

AI Agent Orchestration Platform — Architecture

1. Design philosophy

Three principles, in priority order, that resolve every downstream decision:

1. Agents are configuration; the runtime is code. An "agent" in the database is a row — a system prompt, a model binding, a tool list, a memory policy. The code that *runs* agents is generic. If adding a new agent requires writing Python, the abstraction is wrong. This is what separates a platform from a hardcoded multi-agent demo.

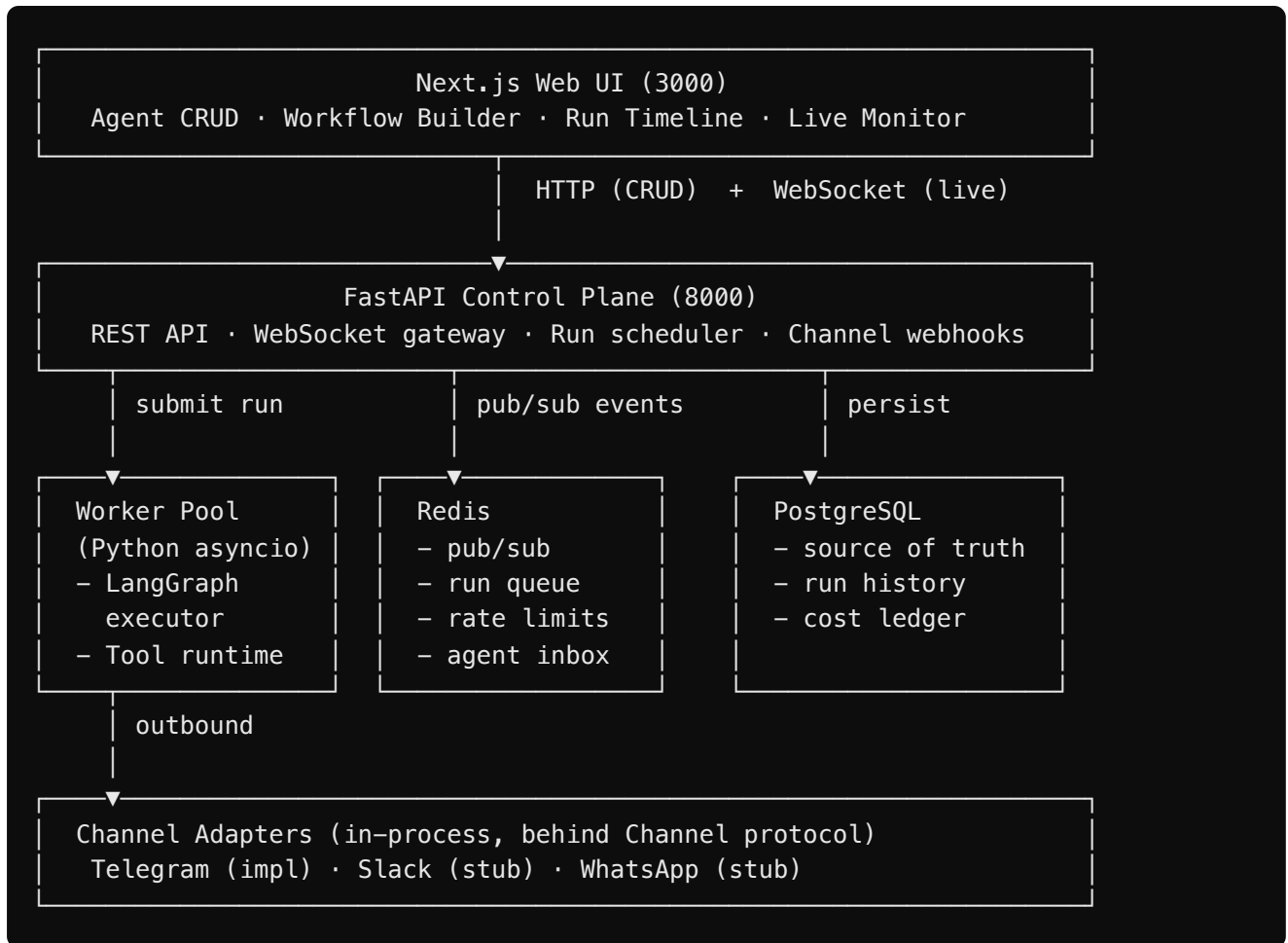
2. Workflows are graphs of agents; agents are graphs of reasoning steps. These are two distinct concerns and must not share an abstraction. Conflating them is the most common failure mode in agent platforms — it produces workflow builders that leak LLM concepts and reasoning loops that leak orchestration concepts.

3. Every side effect is a row. Every LLM call, tool invocation, inter-agent message, and channel event writes to Postgres before anything else happens. This isn't logging — it's the *primary* representation. The UI, the timeline view, the cost tracking, the replay capability all derive from this. Redis is transport, not truth.

If a future decision contradicts one of these, the decision is wrong.

2. System topology

Four processes. One repository. One `docker-compose up`.



Why this shape

Control plane and workers are separate processes. A long-running agent task (web search, LLM call, retry) must not block the API. They communicate through Redis (job queue) and Postgres (state). This is also what makes horizontal scaling trivial later — workers scale independently — but the *real* reason to do it now is so a stuck agent doesn't freeze your UI during the live demo.

Redis is ephemeral, Postgres is canonical. Redis carries pub/sub events for the live UI, the work queue, and rate limits. If Redis dies, in-flight runs fail but no data is lost. This boundary needs to be explicit in the code — the worker writes to Postgres *first*, then publishes to Redis. Never the reverse.

Channels are in-process adapters, not separate services. Webhooks land on the FastAPI control plane, which translates them into internal events. A separate "telegram-worker" container would be over-engineering for the demo and harder to debug.

No message broker beyond Redis pub/sub. RabbitMQ/Kafka would be the "correct" choice for inter-agent messaging at scale. For this scope, Redis pub/sub + Postgres outbox pattern is enough and adds no operational complexity.

3. The two-layer execution model

This is the load-bearing architectural decision. Get this wrong and nothing else matters.

Outer layer: Workflow Graph

A workflow is a directed graph authored by the user in the visual builder. Nodes are *agents*. Edges are *transitions*, optionally guarded by conditions evaluated against the workflow state.

Implementation: LangGraph `StateGraph`. State schema is fixed across all workflows:

```
class WorkflowState(TypedDict):
    run_id: UUID
    messages: list[AgentMessage]      # full conversation across agents
    artifacts: dict[str, Any]          # named outputs from agents
    current_agent: str | None
    iteration_count: int               # for loop termination
    metadata: dict[str, Any]          # cost, tokens, timing accumulators
```

Node execution: Each node receives the state, runs *its agent's inner graph* (next section), and returns updated state. The node is generic — it knows nothing about the agent's specific behavior. It looks up the agent config by ID, instantiates the inner graph, runs it, persists results, returns.

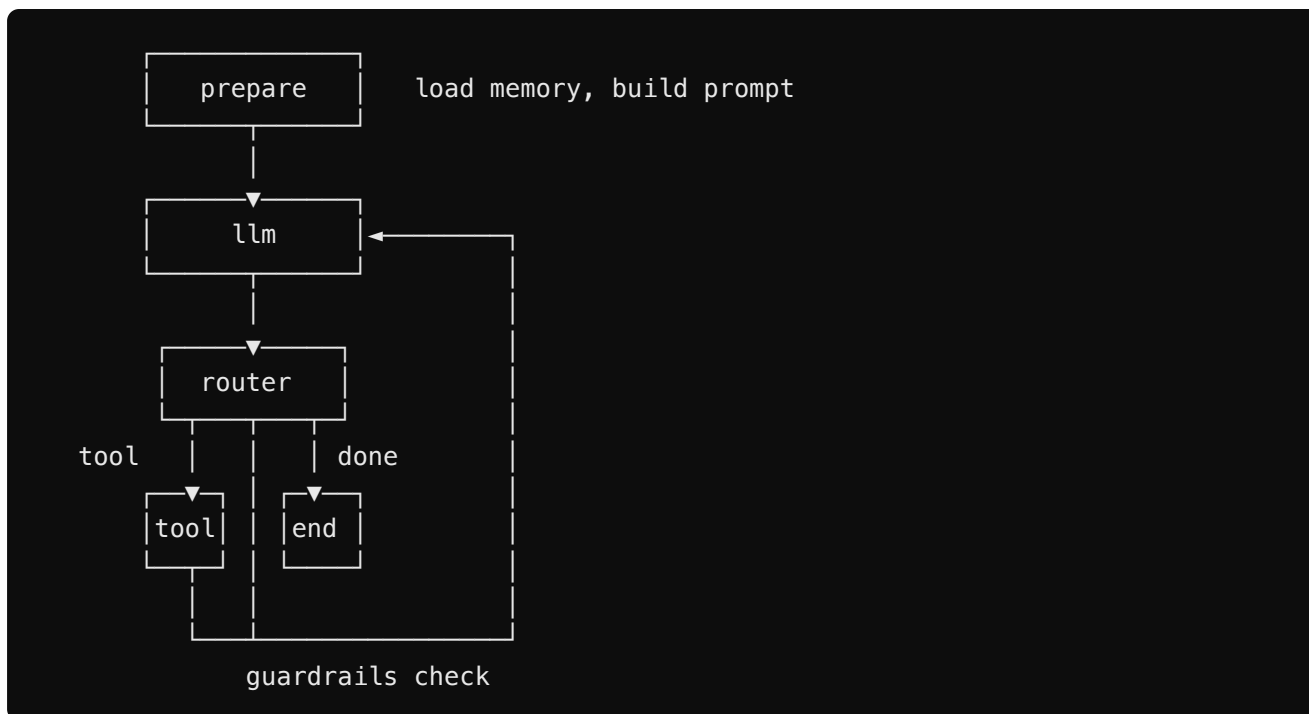
Edge conditions: Authored as constrained expressions, not arbitrary code. A small DSL:

```
artifacts.classification == "refund"
iteration_count < 3
last_message.contains("APPROVED")
```

Parse with a real parser (Lark or pyparsing — *not* `eval`). About 200 lines including AST and evaluator. The reason to do this rather than allow arbitrary Python: the workflow JSON must be safe to load from untrusted sources, replayable, and viewable in the UI. Plus, in the interview, "I built a small expression DSL" is a much better answer than "I exec strings."

Inner layer: Agent Reasoning Graph

When the outer graph invokes an agent, it runs that agent's *inner graph* — a ReAct-style loop:



Why a separate graph instead of a function: Persistence, observability, and termination. Each node transition is a row in Postgres. The router enforces guardrails (max iterations, token caps, tool allowlist) at a single chokepoint. Restartability comes for free — a crashed worker can resume from the last persisted node.

Why not let agents call other agents directly: Tempting, and AutoGen does this. It collapses into spaghetti fast. In this architecture, agent-to-agent communication is *always* mediated by the outer workflow graph, which means: the user authored that interaction in the builder, it's visible in the timeline, and it can be reasoned about. The exception — `send_message_to_agent` as a tool — exists but is constrained (next section).

The exception: dynamic inter-agent messaging

Some workflows genuinely need an agent to decide *at runtime* which other agent to message. The "router agent" pattern requires it. Two options:

- **Option A:** Router agent uses conditional edges in the outer graph. Clean but rigid — every possible recipient must be a graph node.
- **Option B:** `send_message_to_agent` tool with an allowlist of recipients defined in the workflow.

Ship both. Option A for static routing (the common case), Option B for genuinely dynamic dispatch. The tool implementation writes a message row, publishes to the recipient agent's Redis inbox, and the outer graph's scheduler picks it up on the next tick. This is the "async messaging" requirement, properly implemented.

4. Data model

```
-- Identity & configuration
agents (
  id          UUID PRIMARY KEY,
  name        TEXT UNIQUE NOT NULL,
  role        TEXT NOT NULL,          -- short role description
  system_prompt TEXT NOT NULL,
  model_provider TEXT NOT NULL,      -- 'anthropic' | 'openai' | 'bedrock'
  model_name   TEXT NOT NULL,
  temperature  REAL NOT NULL DEFAULT 0.7,
  max_tokens   INT NOT NULL DEFAULT 2048,
  tool_ids     TEXT[] NOT NULL DEFAULT '{}',
  memory_policy JSONB NOT NULL,      -- {strategy: 'buffer'|'summary', ...}
  guardrails   JSONB NOT NULL,      -- {max_iterations, max_cost_usd, ...}
  created_at   TIMESTAMPTZ NOT NULL,
  updated_at   TIMESTAMPTZ NOT NULL
)

workflows (
  id          UUID PRIMARY KEY,
  name        TEXT UNIQUE NOT NULL,
  description  TEXT,
  graph       JSONB NOT NULL,        -- {nodes: [...], edges: [...]}
  template_id TEXT,                  -- nullable, for built-ins
  version     INT NOT NULL DEFAULT 1, -- bump on edit, old runs reference old version
  created_at  TIMESTAMPTZ NOT NULL
)

workflow_versions (                    -- immutable history
  workflow_id UUID,
  version     INT,
  graph       JSONB NOT NULL,
  created_at  TIMESTAMPTZ NOT NULL,
  PRIMARY KEY (workflow_id, version)
)

-- Channels
channels (
  id          UUID PRIMARY KEY,
  type        TEXT NOT NULL,          -- 'telegram' | 'slack' | 'whatsapp'
  name        TEXT NOT NULL,
  config      JSONB NOT NULL,        -- {bot_token: ..., webhook_secret: ...}
  status      TEXT NOT NULL,          -- 'active' | 'disabled' | 'error'
  created_at  TIMESTAMPTZ NOT NULL
)

channel_bindings (
  id          UUID PRIMARY KEY,
  agent_id    UUID REFERENCES agents,
  channel_id  UUID REFERENCES channels,
  external_id TEXT NOT NULL,          -- chat_id, channel_id, etc.
```

```

    config          JSONB NOT NULL DEFAULT '{}',
    UNIQUE (channel_id, external_id)
)

-- Execution
runs (
    id              UUID PRIMARY KEY,
    workflow_id     UUID REFERENCES workflows,
    workflow_version INT NOT NULL,
    status          TEXT NOT NULL,          -- 'pending'|'running'|'completed'|'failed'
    trigger_type    TEXT NOT NULL,         -- 'manual'|'schedule'|'channel'|'agent'
    trigger_payload JSONB,                 -- inbound message, cron expr, etc.
    initial_state   JSONB,
    final_state     JSONB,
    error           TEXT,
    total_tokens_in INT NOT NULL DEFAULT 0,
    total_tokens_out INT NOT NULL DEFAULT 0,
    total_cost_usd  NUMERIC(10,6) NOT NULL DEFAULT 0,
    started_at     TIMESTAMPTZ NOT NULL,
    completed_at    TIMESTAMPTZ,
    INDEX (workflow_id, started_at DESC),
    INDEX (status) WHERE status IN ('pending', 'running')
)

steps (
    id              UUID PRIMARY KEY,          -- one row per outer-graph node execution
    run_id          UUID REFERENCES runs ON DELETE CASCADE,
    agent_id        UUID REFERENCES agents,
    node_id         TEXT NOT NULL,             -- node id from workflow graph
    status          TEXT NOT NULL,
    parent_step_id  UUID REFERENCES steps,     -- for branching/looping visualization
    started_at      TIMESTAMPTZ NOT NULL,
    completed_at    TIMESTAMPTZ,
    tokens_in       INT NOT NULL DEFAULT 0,
    tokens_out      INT NOT NULL DEFAULT 0,
    cost_usd        NUMERIC(10,6) NOT NULL DEFAULT 0,
    error           TEXT,
    INDEX (run_id, started_at)
)

messages (
    id              UUID PRIMARY KEY,          -- every LLM exchange + inter-agent messages
    run_id          UUID REFERENCES runs ON DELETE CASCADE,
    step_id         UUID REFERENCES steps,
    agent_id        UUID,                     -- author; null = system or user
    role            TEXT NOT NULL,             -- 'system'|'user'|'assistant'|'tool'|'agent'
    content         TEXT NOT NULL,
    tool_calls      JSONB,                     -- structured tool invocations
    recipient_agent_id UUID,                  -- for inter-agent messages
    channel_message_id TEXT,                  -- for messages sent to external channels
    tokens_in       INT NOT NULL DEFAULT 0,
    tokens_out      INT NOT NULL DEFAULT 0,
    cost_usd        NUMERIC(10,6) NOT NULL DEFAULT 0,

```

```

latency_ms      INT,
ts              TIMESTAMPTZ NOT NULL,
INDEX (run_id, ts),
INDEX (recipient_agent_id, ts) WHERE recipient_agent_id IS NOT NULL
)

tool_invocations (
  id            UUID PRIMARY KEY,
  message_id    UUID REFERENCES messages ON DELETE CASCADE,
  tool_name     TEXT NOT NULL,
  input         JSONB NOT NULL,
  output        JSONB,
  error         TEXT,
  latency_ms    INT NOT NULL,
  ts            TIMESTAMPTZ NOT NULL
)

schedules (
  id            UUID PRIMARY KEY,
  workflow_id   UUID REFERENCES workflows,
  cron_expression TEXT NOT NULL,
  enabled       BOOLEAN NOT NULL DEFAULT true,
  last_run_at   TIMESTAMPTZ,
  next_run_at   TIMESTAMPTZ NOT NULL,
  payload       JSONB,                      -- initial state for triggered runs
  INDEX (enabled, next_run_at)
)

-- Outbox pattern for reliable channel delivery
outbound_messages (
  id            UUID PRIMARY KEY,
  channel_id    UUID REFERENCES channels,
  external_id   TEXT NOT NULL,              -- recipient
  content       TEXT NOT NULL,
  status        TEXT NOT NULL,              -- 'pending'|'sent'|'failed'
  attempts      INT NOT NULL DEFAULT 0,
  last_error    TEXT,
  created_at    TIMESTAMPTZ NOT NULL,
  sent_at       TIMESTAMPTZ,
  INDEX (status, created_at) WHERE status = 'pending'
)

```

Key design choices:

- **workflow_versions** is immutable. When a user edits a workflow, the old version is preserved. Runs reference the version they used. This means run history remains coherent across workflow edits — without it, the timeline view lies after any edit.
- **steps** table exists separately from **messages**. A step is a node execution; a message is an LLM exchange. One step contains multiple messages. Without this split, the timeline view requires painful aggregation queries.

- **Costs are denormalized up the hierarchy** (`messages.cost_usd` → `steps.cost_usd` → `runs.total_cost_usd`). Computed at write time, not query time. The dashboard queries stay simple.
- **Outbox pattern for channel sends.** When an agent decides to send a Telegram message, the worker writes to `outbound_messages` in the same transaction as the `messages` row. A separate dispatcher reads pending rows and calls the channel API. This guarantees that either both happen or neither — no "the agent thinks it sent, but Telegram never got it." Standard distributed systems hygiene; worth the 50 lines.

5. The Channel abstraction

```
class Channel(Protocol):
    """A bidirectional messaging integration."""

    type: str # 'telegram' | 'slack' | ...

    async def initialize(self, config: dict) -> None:
        """Set up webhooks, validate credentials. Idempotent."""

    async def send(
        self,
        external_id: str,
        content: str,
        metadata: dict | None = None
    ) -> str:
        """Send a message. Returns provider message id."""

    def parse_webhook(self, headers: dict, body: bytes) -> InboundMessage | None:
        """Parse and verify a webhook payload. Returns None if invalid."""

    async def health_check(self) -> ChannelHealth:
        """Verify the channel is operational."""

    @dataclass
    class InboundMessage:
        channel_id: UUID
        external_id: str # sender chat id
        content: str
        sender_name: str | None
        attachments: list[Attachment]
        raw: dict # provider-specific payload, preserved
        received_at: datetime
```

Webhook flow:

1. FastAPI route `POST /webhooks/{channel_id}` receives the request
2. Loads `Channel` by id, calls `parse_webhook` — verifies signature, parses payload

3. If valid, looks up `channel_bindings` for the `external_id` to find the bound agent
4. Resolves to a workflow (each binding can specify a workflow, or use the agent's default)
5. Enqueues a run with `trigger_type='channel'` and the inbound message as initial state
6. Returns 200 immediately — processing is async

Why this layering: The control plane never imports `python-telegram-bot`. It calls `channel.send()`. When the Slack adapter is added, the only changes are: implement `SlackChannel`, register in `ChannelRegistry`, no orchestrator changes. In the interview, you can prove this by showing the diff.

Concrete extensibility test in the README: "To add Discord support: implement `DiscordChannel(Channel)` in `channels/discord.py`, register it in `channels/__init__.py`, add a config form in the UI. No changes to orchestrator, agents, or workflows." That's a much stronger claim than hand-waving about extensibility.

6. Memory subsystem

```
class MemoryStrategy(Protocol):
    async def load(self, agent_id: UUID, context: MemoryContext) -> list[Message]:
        """Return messages to inject into the agent's prompt."""

    async def save(self, agent_id: UUID, messages: list[Message]) -> None:
        """Persist after a turn completes."""

@dataclass
class MemoryContext:
    run_id: UUID
    channel_external_id: str | None # for cross-run memory keyed to a user
    max_tokens: int
```

Built-in strategies:

- **BufferMemory** — last N messages within the current run. Default. No cross-run memory.
- **SummaryMemory** — last K messages verbatim + rolling summary of older context. Summary update is triggered when the window exceeds a threshold; a small LLM call rewrites the summary.
- **ChannelScopedMemory** — buffer or summary, but scoped to a `channel_external_id` rather than `run_id`. This is how a Telegram bot remembers a user across separate runs.

Memory is queried, not pushed. The agent's `prepare` node calls `memory.load(...)`. The strategy decides what to return. This keeps the LLM call site dumb and centralizes memory logic.

Future-work hook (extremis): A fourth strategy, `ExternalMemoryStrategy`, talks to an MCP-compatible memory server — episodic recall by semantic similarity, identity layer for personality persistence, procedural memory for learned tool sequences. In this build it is wired for real

against extremis as a selectable per-agent memory policy. The strategy interface is the seam; swapping the backend changes nothing above it.

7. Tool runtime

```
class Tool(Protocol):
    name: str
    description: str
    input_schema: dict          # JSON Schema, sent to the LLM
    requires_approval: bool     # for guardrails

    async def execute(self, input: dict, context: ToolContext) -> ToolResult

@dataclass
class ToolContext:
    agent_id: UUID
    run_id: UUID
    step_id: UUID
    # constrained capabilities – tools cannot reach beyond these
    db: ReadOnlySession        # tools can read state, not write directly
    http: AllowlistedClient    # outbound HTTP through a filtered client
    budget: BudgetTracker      # token/cost limits per invocation
```

Why a context object instead of globals or DI: Tools must be testable in isolation, must not have ambient authority, and must report their resource usage. The context is the single seam.

Tools shipped:

Tool	Purpose	Lines (est.)
<code>web_search</code>	Tavily API search	~80
<code>http_request</code>	Allowlisted HTTP GET/POST	~120
<code>send_message_to_agent</code>	Inter-agent messaging via outbox	~100
<code>python_exec</code>	Sandboxed subprocess (separate container)	~200
<code>send_to_channel</code>	Outbound channel message	~60

`python_exec` **deserves a note.** Running arbitrary LLM-generated Python in your worker process is dangerous. Three options:

- **Don't ship it** — safest, less impressive
- **Subprocess with `nsjail` or `firejail`** — proper sandboxing, Linux-only, adds dep
- **Separate Docker container with no network, ephemeral volume, ulimits** — what to ship. The worker calls into a `code-runner` container over a Unix socket. ~50 lines of Docker config, isolation is robust enough for a local demo, doesn't require host-specific security tools.

This shows up well in the architecture diagram and in the interview as "I thought about the security boundary."

8. Guardrails

Configured per-agent in `agents.guardrails`, enforced at the inner graph's router node:

```
class Guardrails(BaseModel):
    max_iterations: int = 10          # ReAct loop limit
    max_tokens_per_turn: int = 8000
    max_cost_per_run_usd: Decimal = Decimal("1.00")
    max_tool_calls_per_turn: int = 5
    allowed_tools: list[str] | None = None # override agent.tool_ids if set
    require_approval_for: list[str] = []  # tools that pause for human approval
    pii_redaction: bool = False          # outbound message scrubbing
    output_max_length: int = 10000       # truncate runaway outputs
```

Enforcement model: A `GuardrailEnforcer` is invoked at every router transition. It can return `CONTINUE`, `TERMINATE`, or `PAUSE_FOR_APPROVAL`. The state machine respects this — guardrails are not advisory.

PII redaction is a stub for the demo (regex for emails/phones), but the hook is there. Mention in the README that production would use a Presidio integration or similar.

Cost circuit breaker is the one guardrail you should demo. Set a run's max cost to \$0.10, watch the workflow terminate gracefully when exceeded, with a proper error in the timeline. This is the kind of detail that separates "I built a multi-agent thing" from "I built infrastructure."

9. The UI

Three primary surfaces. Build them in this priority order — if anything slips, the third is the one to cut, not the first.

9.1 Agent & Workflow Configuration

Standard CRUD. Two non-obvious choices:

- **Schema-driven forms.** Agent config and guardrails are JSONB in the database. Use a JSON Schema → form library (RJSF or similar) so adding a new config field is a backend schema change with zero frontend work. This is what makes the system actually configurable, as the rubric demands.
- **Versioned workflow edits.** Saving a workflow creates a new `workflow_versions` row. The UI shows version history. Running an old version is a one-click action. This is the kind of feature that's invisible until someone needs it, then critical.

9.2 Visual Workflow Builder

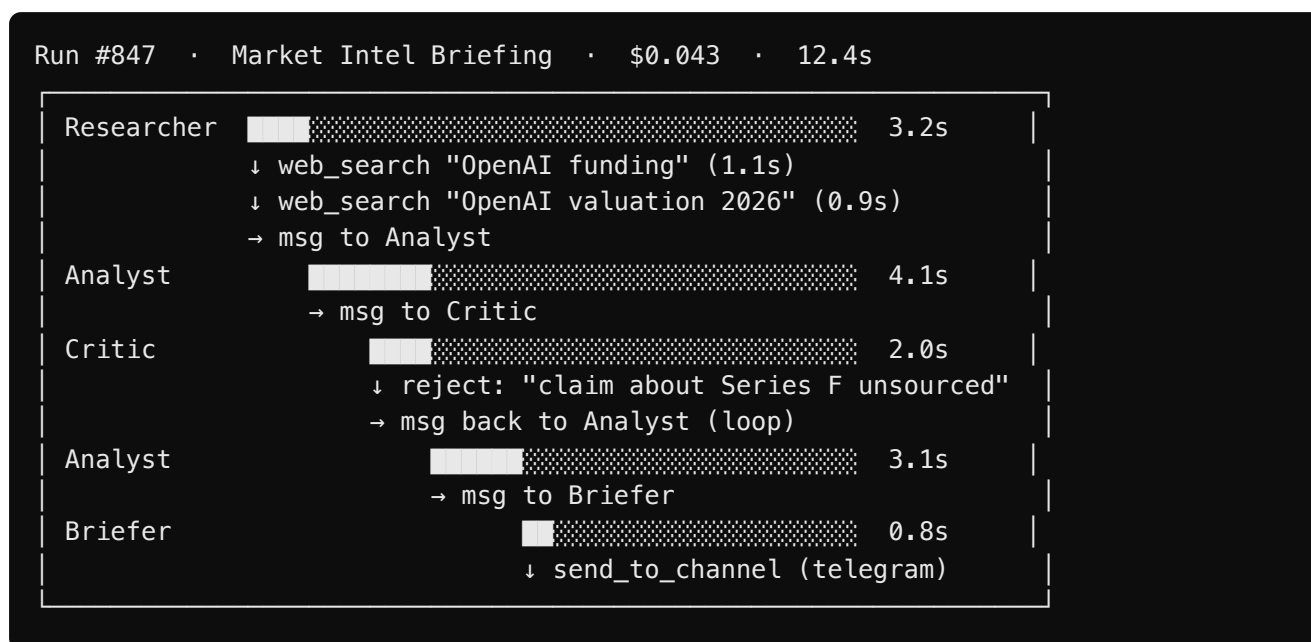
React Flow. Custom node types per agent role. Edges carry condition expressions edited in a side panel with syntax highlighting (CodeMirror with the DSL grammar). Save serializes to the `graph` JSONB schema.

Three features to include, because they're cheap and dramatically improve perception:

- **Live validation** — disconnected nodes, unreachable agents, cyclic edges without termination conditions. Show inline warnings.
- **Dry-run mode** — submit a workflow with a test input, get the execution path highlighted on the graph in real time (this is the WebSocket payoff).
- **"Inspect node"** — click a node, see the agent config in a side panel without leaving the canvas.

9.3 Run Timeline & Live Monitor

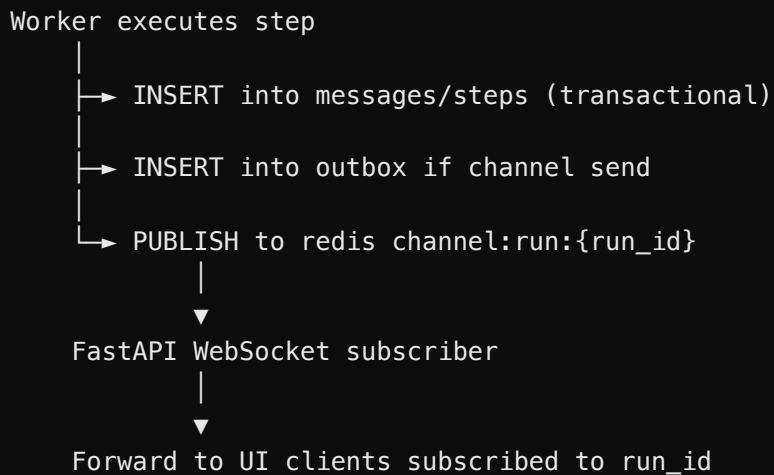
This is the demo-defining surface. Not a log viewer — a *visualization*.



Each agent is a row. Each step is a block scaled to duration. Tool calls and inter-agent messages overlay as annotations. Hovering shows tokens, cost, latency, and the message content. Clicking opens the full message thread for that step.

Live updates via WebSocket — as a run progresses, blocks animate in. This is the moment in the demo video that earns the architecture grade. About 2-3 days of work to do well; worth every hour.

10. Real-time event flow



Design constraint: the worker must never publish to Redis before the Postgres write commits. If Redis publishes succeed but Postgres rolls back, the UI shows phantom events. Use Postgres `LISTEN/NOTIFY` instead? Tempting, but it doesn't survive worker restarts cleanly. The transactional outbox pattern (with a dispatcher polling for unpublished rows) is the robust answer. For this scope, the simpler "commit then publish" with idempotent UI handling is acceptable — document the trade-off in the README.

Backpressure: WebSocket clients that fall behind get dropped after a buffer threshold. They reconnect and replay from `messages` table by `run_id` + `ts > last_seen`. This is why every event also lives in Postgres — replay is free.

11. Code organization

```
/
├── docker-compose.yml
├── README.md
├── docs/
│   ├── architecture.md           # this document
│   ├── adding-a-channel.md
│   ├── adding-a-tool.md
│   └── workflow-dsl.md
├── backend/
│   ├── pyproject.toml
│   ├── alembic/                  # migrations
│   ├── app/
│   │   ├── main.py              # FastAPI entrypoint
│   │   ├── config.py            # pydantic-settings
│   │   ├── db/
│   │   │   ├── models.py        # SQLAlchemy
│   │   │   ├── session.py
│   │   │   └── repositories/    # per-aggregate data access
│   │   ├── api/
│   │   │   ├── agents.py
│   │   │   ├── workflows.py
│   │   │   ├── runs.py
│   │   │   ├── channels.py
│   │   │   ├── webhooks.py
│   │   │   └── ws.py
│   │   ├── runtime/
│   │   │   ├── outer_graph.py   # workflow execution
│   │   │   ├── inner_graph.py   # agent reasoning loop
│   │   │   ├── state.py         # WorkflowState, AgentState
│   │   │   ├── executor.py      # worker entrypoint
│   │   │   └── dsl/             # condition expression parser
│   │   ├── channels/
│   │   │   ├── base.py          # Channel protocol
│   │   │   ├── telegram.py
│   │   │   ├── slack.py         # stub
│   │   │   ├── whatsapp.py      # stub
│   │   │   └── registry.py
│   │   ├── tools/
│   │   │   ├── base.py
│   │   │   ├── web_search.py
│   │   │   ├── http_request.py
│   │   │   ├── send_to_agent.py
│   │   │   ├── send_to_channel.py
│   │   │   ├── python_exec.py
│   │   │   └── registry.py
│   │   └── memory/
│   │       ├── base.py
│   │       ├── buffer.py
│   │       └── summary.py
```

```

├── channel_scoped.py
├── guardrails/
│   ├── enforcer.py
│   └── policies.py
├── schedules/
│   └── scheduler.py      # APScheduler integration
├── observability/
│   ├── cost.py          # provider pricing tables
│   └── events.py        # redis pub/sub helpers
├── templates/
│   ├── market_intel.py
│   └── personal_assistant.py
├── tests/
│   ├── unit/
│   ├── integration/
│   └── fixtures/
├── frontend/
│   ├── package.json
│   ├── next.config.js
│   ├── app/
│   │   ├── layout.tsx
│   │   ├── page.tsx      # dashboard
│   │   ├── agents/
│   │   ├── workflows/
│   │   ├── runs/
│   │   └── channels/
│   ├── components/
│   │   ├── workflow-builder/ # React Flow + DSL editor
│   │   ├── timeline/        # run visualization
│   │   ├── agent-config/    # schema-driven forms
│   │   └── ui/              # shadcn primitives
│   ├── lib/
│   │   ├── api.ts          # typed API client
│   │   └── ws.ts           # WebSocket subscriber
│   └── types/              # generated from backend OpenAPI
├── code-runner/            # sandboxed python_exec container
│   ├── Dockerfile
│   └── runner.py
├── scripts/
│   ├── seed.py            # creates templates + sample agents
│   └── reset_db.py

```

Why this layout matters:

- **runtime/** is the heart of the system. It must be readable in isolation. No FastAPI imports, no HTTP concerns. Just: given an agent config and an input, run it.
- **repositories/** instead of querying from API handlers. API handlers stay thin; data access stays testable; if you later swap Postgres for something else, one directory changes.

- **Types generated from OpenAPI.** Backend defines pydantic models, exports OpenAPI, frontend `openapi-typescript` generates TS types. No hand-maintained DTOs.

Note: the build adds two further top-level modules beyond this base tree — `backend/app/harness/` (the unified harness layer) and the workflow-creation surfaces. See `docs/plan.md` for those subsystems and the multimodal/image handling.

12. Testing strategy

The rubric says "tests for critical paths." Don't write 500 unit tests; write the *right* tests.

Three integration tests that exercise the system end-to-end:

1. `test_agent_lifecycle` — create agent via API, configure tools and memory, retrieve via API, update, verify version increment. Catches CRUD/schema bugs.
2. `test_workflow_execution` — load a fixture workflow with 2 mocked agents (deterministic LLM responses via a stub provider), trigger a run, assert: all steps written, messages persist in order, total cost computed correctly, run reaches `completed`. This single test catches 80% of regressions.
3. `test_channel_roundtrip` — POST a simulated Telegram webhook, assert: webhook parsed, binding resolved, run triggered, outbound message written to outbox, dispatcher would send it (mock the actual API). Catches the channel wiring.

Unit tests for the things that benefit from them:

- DSL expression parser (table-driven, ~20 cases — your favorite)
- Cost calculator across providers (subtle off-by-one bugs love this)
- Guardrails enforcer (one test per policy)
- Memory strategies (buffer eviction, summary trigger thresholds)

Skip:

- Mocking LangGraph internals
- Testing FastAPI route signatures (the framework handles that)
- 100% coverage targets

A single golden-path E2E test in Playwright: create an agent in the UI, run a workflow, verify the timeline renders the expected steps. This catches frontend/backend contract drift and is the test you can demo if asked "how do you verify the system works end-to-end?"

13. Observability beyond the UI

The UI gives the user observability. The *operator* (you, during the demo) needs more.

- **Structured logging** with `structlog`. Every log line gets `run_id`, `step_id`, `agent_id` automatically through context binding. When something breaks in the demo, you grep one ID and see the whole story.
- **OpenTelemetry traces** wrapping each step. Export to a local Jaeger container in docker-compose (optional, behind a flag). Mention in README — production would point this at any OTLP-compatible backend.
- **Health endpoints** — `/health`, `/health/db`, `/health/redis`, `/health/channels`. The UI shows channel status using these.

This is also where you mention `peekr` in the README as the LLM-specific observability layer you'd add — auto-instruments the LLM clients. Don't integrate it for the demo (scope), but the docs say "the cost tracking system is intentionally compatible with peekr's tracing model."

14. Failure modes and how the system handles them

Worth a section in the README and a question you'll absolutely get.

Failure	Detection	Response
Worker crash mid-run	Heartbeat row missing > 30s	Mark run <code>failed</code> , surface in UI with restart button
LLM provider timeout	HTTP timeout in client	Retry with exponential backoff (max 3); on final failure, fail the step, optionally route to fallback model per agent config
Tool error	Exception in tool execute	Capture in <code>tool_invocations.error</code> , return error to LLM, let it decide whether to retry or give up
Channel webhook signature invalid	Verification fails	Return 401, log, no run created
Postgres down	Connection error	Control plane returns 503, workers retry; nothing is lost because no one can write anything
Redis down	Pub/sub disconnect	Runs still execute (writes to Postgres still work), live UI degrades to polling-from-Postgres mode
Guardrail violation (cost cap)	Enforcer at router node	Terminate run gracefully, write <code>runs.error</code> , surface in timeline as red block
Agent infinite loop	Iteration counter at router	Terminate, write error, surface in timeline
Outbox send failure (Telegram API down)	Exception in dispatcher	Increment <code>attempts</code> , exponential backoff, after N attempts mark <code>failed</code> and surface in UI

The fact that this table exists at all is the signal. "I thought through what breaks" is the architectural maturity they're checking for.

15. Security posture (for the local demo)

You're not deploying this to the internet, but the questions will come.

- **Secrets** — `.env` for local, `pydantic-settings` for loading. Never logged. README has a clear `.env.example`.
- **Channel webhooks** — signature verification per provider (Telegram secret token, Slack signing secret). Reject unverified requests at the FastAPI layer.
- **Tool sandboxing** — `python_exec` in a separate container as discussed. `http_request` has an allowlist of domains configured per-agent. No `file_system` tool ships at all.
- **Prompt injection** — explicit "this content is from an external user" prefix when injecting channel messages into prompts. Not bulletproof, but the right hook. Mention as known limitation.
- **Database access** — connection-pooled, read-only roles for tools that need DB access. The control plane has full write access; workers have a more limited role (no DDL).
- **No auth for the local UI**, by design — single-user app. README notes that production would add NextAuth + per-user resource scoping.

16. The README architecture

Since 10% of the grade is documentation and the interviewer will read it first:

README.md

- |— What this is (1 paragraph)
- |— 60-second demo (gif embedded)
- |— Quickstart (5 commands max to a working system)
- |— Architecture overview (the topology diagram + 3 paragraphs)
- |— Why these choices
 - |— Why LangGraph
 - |— Why two-layer execution
 - |— Why Postgres + Redis (no broker)
 - |— Why local-first
- |— Harness layer (modes table, add-a-provider, recording, eval)
- |— How to add things
 - |— A new agent (UI walkthrough)
 - |— A new tool (code example)
 - |— A new channel (code example, points at slack.py stub)
 - |— A new workflow template (code example)
- |— What's in scope vs. cut (this list itself is a signal)
- |— Failure modes (the table above)
- |— Future work
 - |— Multi-tenancy & auth
 - |— External memory via extremis (wired)
 - |— peekr integration for LLM observability
 - |— Production deployment notes
 - |— Multi-model routing
- |— Live demo script (what to expect in the recording)

The "what's in scope vs. cut" section is the one most candidates won't write. It demonstrates judgment. "I cut X, Y, Z because of time, here's how I'd add them" is the answer to half the live-session questions before they're asked.

17. The single biggest risk

Not technical. **The demo recording.** A working system with a confusing video loses to a slightly-less-working system with a crisp video. Plan it:

- Script the demo before building. Know the exact narrative arc: "Here's the dashboard → create an agent → wire two agents in the builder → trigger via Telegram → watch the timeline → show cost tracking → show conditional loop in Template 2."
- Record by day 10 of 14, not day 14. The harness `replay` mode exists precisely to make this deterministic and offline.
- Two takes minimum. The second one is always cleaner.

If a feature doesn't earn screen time in the demo, question whether it earns implementation time.

This document is the canonical architecture reference. The execution plan, plus the workflow-creation, unified-harness, multimodal, and frontend-design subsystems, live in `docs/plan.md`.